Final technical summary

License plate detection and recognition on CCPD2019

Final submission, 2 November 2026

Code: `https://github.com/S1norin/inno-detecting-license-plates`

Contributors

Name	Email
Mikhail Krylov	m.krylov@innopolis.university
Maximilian Mifsud Bonici	m.mifsudbonici@innopolis.university
Andrei Polevoi	a.polevoi@innopolis.university
Evgenii Khovalyg	e.khovalyg@innopolis.university

1 Task, dataset, and system

We detect and recognize Chinese license plates on a 5,000-photo CCPD2019 subset expanded to 15,000 images (scale variants 1.0/0.75/0.5, 30 tilt angles). Splits are assigned *before* resizing, so all variants of one photo stay in one split: 3,500/700/800 train/val/test per scale. Size buckets (plate width, train-derived): small ≤ 83 px, medium ≤ 112 px, large > 112 px; tilt bins 0–9/10–19/20°. Primary metrics use the original-scale split; the 0.75/0.5 variants are evaluated in Section 4.

The system pairs a YOLO11s detector with a 3.0M-parameter grayscale CRNN (CNN + BiLSTM + CTC, 59 classes) recognizer. Detector variants: **frozen** (off-the-shelf `yolo11s-license-plate`) and **fine-tuned** (same backbone, 3,500 train images, 640 px, seed 42, RTX 3090 Ti, 1,439.8 s; best epoch 4 of 24). The CRNN was trained on the 3,500 original-scale train crops (50 epochs, 39 selected, CPU, 4,154.9 s). A frozen EasyOCR baseline (committed numbers) provides the off-the-shelf reference. All runs: shared harness, CPU single-threaded, IoU 0.5, conf 0.001, NMS 0.7, 640 px, seed 42, digest-verified weights. Recognition uses ground-truth crops (`ocr_only`) or the detector's top-1 boxes (`end_to_end`); mode and abstention threshold are selected on validation only, then test is run once.

2 Detection (800 test images)

Model	Split	mAP@0.5	Precision	Recall	F1
Frozen YOLO11s	val	0.9245	0.9402	0.8986	0.9189
Frozen YOLO11s	test	0.9073	0.9408	0.8938	0.9167
Fine-tuned YOLO11s	val	0.9949	0.9549	0.9986	0.9763
Fine-tuned YOLO11s	test	0.9949	0.9466	0.9975	0.9714

Table 1: Detection; the fine-tuned model was cross-checked with native Ultralytics validation (identical mAP to 1e-4).

The gain from fine-tuning is dominated by recall (0.8938 → 0.9975): the frozen model misses 51 of 800 test images entirely (mostly large tilted plates), the fine-tuned model none. Its weakest scope is the 10–19° tilt bin (F1 0.9254).

3 Recognition (800 test plates, mode `none`, tau 0.5)

Engine	Protocol	Char. acc.	Exact plates	F1
EasyOCR (frozen)	ocr_only	0.3474	0 / 800	—
EasyOCR (frozen)	end_to_end	0.3240	0 / 800	—
CRNN	ocr_only	0.7593	267 / 800	0.7271
CRNN	end_to_end + frozen	0.6710	148 / 800	0.5676
CRNN	end_to_end + fine-tuned	0.7661	296 / 800	0.7372

Table 2: Recognition on the test split.

Preprocessing ablation (validation char. acc.): `none` 0.7558, `gray_contrast` 0.7469, `canny` 0.4575 — no preprocessing wins; Canny destroys glyph information. With the fine-tuned detector, end-to-end accuracy (0.7661) slightly *exceeds* the ground-truth-crop number (0.7593): detector boxes no longer hurt recognition. Weakest recognition scopes (ocr_only): large plates (0.5658, exact rate 0.1069) and the 10–19° bin (0.5193, exact rate 0.0000).

4 Scale robustness

The 0.75/0.5 variants (800 test images/crops each) were evaluated with the same weights and selection policy; nothing was retrained.

	1.0	0.75	0.5
Detection (mAP@0.5 / F1)			
Frozen YOLO11s	0.9073 / 0.9167	0.9218 / 0.9271	0.9294 / 0.9298
Fine-tuned YOLO11s	0.9949 / 0.9714	0.9949 / 0.9726	0.9949 / 0.9708
CRNN test character accuracy			
ocr_only	0.7593	0.7613	0.7610
end_to_end + fine-tuned	0.7661	0.7629	0.7555
end_to_end + frozen	0.6710	0.6713	0.6721

Table 3: Metrics by image/crop scale (test).

Both detectors are flat across scales (frozen mAP 0.907 → 0.930; fine-tuned 0.9949 everywhere) and CRNN character accuracy is flat, because the 48×256 crop resize normalizes away absolute image scale. The loss appears in per-variant size buckets: at 0.5 scale the medium/large buckets fall to 0.44/0.34 character accuracy while small stays at ∼0.80. The frozen detector's end-to-end gap to ocr_only (∼0.09) is constant at every scale; fine-tuning removes it at every scale.

5 Runtime (Apple M3, CPU, 95 timed samples, seed 42)

Stage	Frozen detector	Fine-tuned detector
Detection, median / p95	77.8 / 88.0 ms	80.2 / 98.8 ms
CRNN recognition, median / p95	5.6 / 6.3 ms	5.7 / 6.6 ms
Peak RSS	∼508 MB	∼498 MB

Table 4: Single-threaded latency; preprocessing adds ∼0.4 ms. A full end-to-end image takes ∼83–86 ms.

6 Failure analysis

Representative failures from the regenerated test caches:

Sample	Observation	Likely cause
`6f3c772c…_s100` (frozen)	Large plate, tilt 19°, zero predictions	Weak prior for large plate aspect ratios; 10–19° is the frozen model's weakest tilt bin
`356b4ca6…_s100` (fine-tuned)	11 boxes: 1 true (0.84) + 10 false positives (0.18, 0.06, …)	Fine-tuning removed misses but left low-confidence false positives on cluttered scenes
`261f3b72…_s100` (CRNN)	GT 京 `K06350`, predicted 皖 `AF4069`; large, tilt 29°	Hallucination with low per-character confidence (min 0.24); large-plate glyphs outside the training distribution

Table 5: Failures; all involve large and/or 10–19°-tilted plates.

Cross-cutting pattern: detection failures concentrate on **large + tilted** plates (the frozen model's blind spot, largely fixed by fine-tuning), while recognition failures concentrate on the **same**

large + 10–19° region, where the recognizer was effectively untrained. Both point at the same data gap: the dataset is dominated by small, near-0° plates.

Limitations

EasyOCR numbers are a frozen baseline from an earlier machine (indicative, not a comparable run); fine-tuned YOLO training was done on a GPU and only inference was rerun locally; the timing probe measures single-threaded latency, not throughput. Next steps: a tilt/size-balanced training set and a taller CRNN input for large plates.

References

- Consolidated results with reproduction commands: `docs/md/final_results.md`
- Detection baseline: `baseline/detection/README.md`; fine-tuning: `models/yolo11_finetune/REPORT.md`
- CRNN recognizer: `models/crnn/REPORT.md`; recognition baseline: `baseline/ocr/README.md`
- Repository: `https://github.com/S1norin/inno-detecting-license-plates`